

GitHub Copilot

Token Optimization Under the New Billing

Internal session

June 2026

Office training pack

Agenda

- What changed on 1 June 2026
- The five VS Code Copilot surfaces and what each costs
- Four cost levers, in priority order
- The two-lane workflow
- Quick wins to adopt this week

Time: 20 minutes plus Q&A.

What changed

What changed on 1 June 2026

- **Old system:** one chat turn = one “premium request”, multiplied by a per-model factor. Plans gave us **N requests per month**.
- **New system:** plans give us a **dollar budget in AI Credits**. Every paid surface bills against it **by tokens × per-model rate**.

Three things to remember

- **1 AI Credit = \$0.01**
- Code completion and Next Edit Suggestions remain **free**
- Chat, Edit mode, Agent mode, Code Review are **paid**

Why this matters

A realistic Agent-mode session on Opus can spend **\$0.50 to \$5+** in credits.

Public reports document developers burning **\$30–40 in a single morning** on long agentic sessions.

Pro is **\$10/month**. The math is unforgiving if we are not deliberate.

The good news

The highest-frequency Copilot interactions — inline completion and Next Edit Suggestions — remain **unmetered**. Most of the productivity we get from Copilot is in the free lane.

Plan allowances

Plan	Monthly cost	Monthly AI Credits
Copilot Pro	\$10	\$10
Copilot Pro+	\$39	\$39
Copilot Business	\$19 / seat	\$19 / seat
Copilot Enterprise	\$39 / seat	\$39 / seat

No rollover. Overage billable per token (admin policy applies).

The surfaces

The five surfaces in VS Code

Surface	Bills?	Typical cost shape
Code completion (ghost text)	✓ no	Free
Next Edit Suggestions	✓ no	Free
Chat — Ask	✗ yes	Small per turn
Chat — Edit	✗ yes	Medium — whole file in & out
Chat — Agent	✗ yes	Largest — tool output stacks
Code Review (PR / inline)	✗ yes	Per review, $\propto$ diff size

Where the tokens come from

On every paid turn:

```
cost = (prompt + attached context + chat history
        + tool output history)      x  input_rate
      + (model reply + tool calls)  x  output_rate
```

Four controllable levers, in priority order:

1. **Which model** — 10× spread between cheapest and most expensive
2. **Tool output history** — agent mode only, but dominates totals there
3. **Attached context size** — #selection vs @workspace vs dragged folder
4. **Chat history length** — new chat per topic

The levers

Lever 1 — Model selection

The **single highest-leverage decision** you make per turn.

Tier	Examples	When to use
Cheap	Haiku 4.5, GPT-5 mini, Gemini 3 Flash	Exploration, summarising, simple edits
Standard	Sonnet 4.6, GPT-5.4, Gemini 3.1 Pro	Multi-file edits, careful implementation
Premium	Opus 4.7, GPT-5.5	Hard reasoning, debugging, architecture

Action: in VS Code, set your default chat model to a **cheap-lane** model. Escalate deliberately, not reflexively.

The two-lane workflow

```
EXPLORE  -> Cheap lane (Haiku, GPT-5 mini)
           ask . plan . summarise . iterate freely
           |
           v
EXECUTE   -> Premium lane (Sonnet+ / Opus)
           one well-formed implementation turn
           |
           v
VERIFY    -> Cheap lane again
           review the diff . write tests
```

Framing: **expensive models for executing locked decisions; cheap models for exploring undecided ones.**

Lever 2 — Context size

Every attached token is paid input **on every turn it stays in the conversation.**

Cost ranking, cheapest to most expensive:

```
#selection < #editor < #file:<name> < @workspace < dragged  
folder
```

Rules of thumb:

- 20 lines matter? `#selection` them.
- One file matters? `#file:` it — not `@workspace`.
- `@workspace` is for “I don’t know which files matter yet.”
- **Never drag a folder** unless that is truly the intent.

Lever 3 — Conversation hygiene

Anti-pattern — prompt-chipping:

```
"Refactor this to use a dict."  
"Actually use defaultdict."  
"Make the keys strings."  
"Add type hints."
```

Four turns, four bills, each carrying the whole history.

Pattern — one complete brief:

```
"Refactor this to use a  
collections.defaultdict[str, list[Foo]],  
add type hints, keep the existing public signature."
```

One turn. One bill. Often better output too.

Lever 4 — Agent mode discipline

Agent mode is where the budget vanishes if you are casual.

Front-load the brief:

- **Task** — one sentence
- **In scope / Out of scope** — which files; what NOT to touch
- **Constraints** — patterns to preserve, libs required/forbidden
- **Definition of Done** — how the agent knows to stop
- **Reporting** — summarise files touched and assumptions

A well-shaped brief routinely **cuts agent sessions from 20 turns to 4–6.**

Watch the token counter

The chat panel shows the running token usage in the **bottom-right corner**.

Tokens in context	What to do
Under 30K	Comfortable, continue
30–80K	Expensive on premium — check whether you are going in circles
80K+	Stop. Summarise to a fresh chat.

Reusable context & habits

Reusable context lives in files

Stop re-pasting “remember, we follow X convention”. Put it in a file Copilot auto-loads.

At repo root — `.github/copilot-instructions.md`:

- We use Python 3.12, type hints required.
- No bare ``except``. Use specific exceptions.
- SQLAlchemy 2.x -- no Query API; use `select()`.
- Tests in ``tests/``, one file per source module.

For repeated task shapes — `.github/prompts/<name>.prompt.md`: reusable templates for “write tests in our style”, “refactor to our pattern”.

Quick wins — adopt this week

- ✓ **Default model** → Haiku 4.5 or GPT-5 mini in VS Code
- ✓ **Bind a shortcut** → “New Chat” for one-keystroke restart
- ✓ **.github/copilot-instructions.md** in every active repo
- ✓ **Stop drag-folder** entirely; always `#file:` or `@workspace`
- ✓ **One brief, not four chips** — write the full prompt
- ✓ **Glance at the token counter** before big turns

What we are trading

These habits feel like upfront work.

The trade is **upfront thinking** vs. **downstream burn**:

Habit	Cost
Writing a complete prompt	+1 min thinking
Switching models mid-task	+5 sec clicks
Starting a new chat per topic	+2 sec
Effect on credit burn	−50% to −80% typical
Effect on output quality	Usually meaningfully better
Effect on rework	Lower

Summary

Summary — the five rules

1. **Free is free** — completion and NES don't bill. Default to them whenever the task is shaped like “complete this”.
2. **Cheap model first** — explore on Haiku or mini; escalate only to execute a locked plan.
3. **Attach less** — `#selection` before `#editor` before `#file:` before `@workspace`.
4. **One task = one chat** — new chat per topic change.
5. **Agent mode = high-stakes mode** — front-load the brief, set the Definition of Done.

Resources we maintain

Everything in this talk is documented in the team folder:

- `README.md` — overview and how to use the pack
- `01-new-billing-model.md` — the billing system, allowances, model rate card
- `02-vscode-surfaces.md` — per-surface deep dive
- `03-optimization-playbook.md` — the full playbook
- `04-cheatsheet.md` — one-page printable reference
- `appendix-sources.md` — every claim, where it came from

Thank you

Questions?

Verified June 2026 against docs.github.com/en/copilot/reference/copilot-billing.